

Vibe Coding Workflow for Rapid Product-Market Fit Discovery

Introduction

In this phase of development, our goal is to rapidly **discover the product use-cases that customers truly want**. We have built a robust infrastructure for a filtered browsing experience, and now we will leverage it to experiment with different user experience ideas. This document outlines a "**vibe coding workflow**" – a fast, iterative prototyping approach using AI-assisted coding – that the CEO and CPO can use to create and demo various webpage transformations. By quickly crafting these prototypes and showing them to pilot customers, we aim to identify which features resonate and achieve product-market fit.

Product Context: Filtered Browsing via Proxy

Our product centers around a **man-in-the-middle (MITM) proxy** that intercepts web traffic to deliver a customized browsing experience. The key components of this system include:

- **Traffic Interception & Caching:** The proxy intercepts HTTP requests. On a user's first visit to a page, the proxy fetches the original page from the upstream source and caches it (e.g. in an S3 bucket). On subsequent visits, the proxy can serve the page from cache instead of fetching it again.
- **Offline Page Modification:** Cached pages can be **modified offline** (outside the live browsing session). We can create multiple transformed versions of a page (stored as separate files/IDs in cache). For example, alongside the original page, we might have: a version with certain content removed, a version with specific elements blurred, a version with added highlights, etc.
- **Dynamic Version Serving:** The proxy checks for a special setting (currently implemented via a browser cookie) to decide which version of the page to serve.
- If the cookie is not set or points to "original," the user gets the unmodified page (from cache or upstream).
- If the cookie specifies a certain transformation (e.g. "Version B"), the proxy will retrieve the corresponding modified page from cache and serve that to the user.
- **Two Browsing Experiences:** As a result of the above, users (and testers) can experience two modes:
 - **First Load:** The original page is fetched and cached (no modifications on first fetch).
 - **Transformed Load:** On subsequent loads with a cookie specifying a version, the proxy serves the pre-modified page variant from cache, giving the appearance of a live transformed website.

This infrastructure is already in place, scalable, and cost-efficient. It provides the foundation for experimenting with different content modifications **without needing to alter the live website in real-time**. We can generate many modified versions offline and serve them on-demand, which is perfect for our product discovery goals.

Objective: Discovering the Right User Experience

Now that the technical foundation exists, the next step is **finding what end-user experience delivers real value to customers**. We have hypotheses about how filtering or augmenting web content might benefit users, but we need to validate these directly with customers.

Key objective: Identify which content modifications or filtering features elicit a “Yes, I want this!” reaction from pilot customers. In practice, this means quickly creating demonstrative versions of a customer’s website (or any target website) with various transformations applied, and then gathering feedback on each.

Important characteristics of this discovery phase:

- **Customer-Defined Success:** Rather than guessing internally, we let customer reactions define what features are desirable. For each prototype experience we show, the customer’s feedback (“I love it” vs. “Not useful” or “Needs change”) will guide us.
- **Speed and Iteration:** We want to create these demo experiences rapidly. It’s acceptable if the prototypes are somewhat manual or not fully automated; what matters is speed and the ability to adjust to feedback quickly. The term *vibe coding* encapsulates this quick, exploratory building approach.
- **Live Feel:** The prototypes should feel like a live website experience to the customer. We’re essentially taking a snapshot of their site and altering it, but presenting it in a way that the customer can click around and experience the changes as if it were live. This greatly enhances the feedback quality, as the experience is tangible.
- **Goal of Validation:** The end goal is to find one or more modified browsing experiences that the customer enthusiastically validates. Once they say, “Yes, I like this; can you make it work for real at scale?”, we know we’ve found a strong direction for the product. Those winning ideas will then be scheduled for full engineering implementation (making the feature real-time, scalable, and automated across all pages).

Team Roles and Skills

Executing this workflow involves collaboration between the technical team and the product team, leveraging each member’s strengths:

- **CTO / Chief AI Officer (Technical Lead):** Sets up the technical infrastructure and provides the tools and APIs needed for page caching and modification. The CTO has deep coding expertise and understands the system end-to-end. In this phase, their main role is to empower others (through documentation and systems) rather than individually building each prototype.
- **CEO and CPO (Product Leads) – Vibe Coders:** These are the primary drivers of the vibe coding workflow. Both have enough technical background and experience with AI-assisted development to build simple applications or scripts (they’ve even deployed apps to AWS with services like Cognito in the past). They will use their product insight plus lightweight coding (with AI help) to create and test different page transformations. Their focus is on the **user experience and utility** of the feature being prototyped, rather than the low-level code details.
- *(Both CEO and CPO are comfortable using AI coding assistants – e.g. tools like Cursor or ChatGPT – hence vibe coding, which means they can describe what they want and let the AI help generate the code.)*

By dividing roles this way, the CTO can ensure the platform and tools are ready and stable, and the CEO/CPO can quickly iterate on product ideas without being blocked by heavy engineering cycles. This makes the product discovery process highly agile and creative.

Tools and Infrastructure for Vibe Coding

To support this workflow, we have a set of tools and components in place:

- **Man-in-the-Middle Proxy & Cache:** As described, this proxy system intercepts requests and serves cached pages. It's configured to allow multiple versions via a cookie value. This proxy is the backbone that lets us show transformed pages seamlessly.
- **Cloud Storage (S3) for Pages:** All original and modified pages are stored in our S3 bucket (or similar cloud storage). Each page version is saved as a separate object (with unique identifier). This allows quick retrieval and also persistence of different versions.
- **HTML Manipulation API:** The CTO will document a set of backend APIs or scripts that the CEO/CPO can call. These might include:
 - *Fetch Page:* Retrieve the HTML of a given URL (either from cache or fresh and then cache it).
 - *Save Page Version:* Save a modified HTML (provided as input) as a new version linked to the original URL.
 - *List Versions:* Enumerate available versions for a URL (to choose among them or display differences).
 - *Utilities:* Possibly helpers like “diff two versions” or “apply global transformations across pages”.
- **Large Language Model (LLM) for Content Transformation:** We have access to an LLM (AI model) that can be used to programmatically alter content. For example, we can feed it the HTML of a page and a prompt like “Remove all negative-sentiment text” or “Blur all images with violence”, and get back a modified HTML. The CEO/CPO will primarily interact with the LLM through coded scripts or AI agent tools to perform these transformations.
- **AI Coding Assistant (Cursor, ChatGPT, etc.):** To facilitate vibe coding, the CEO and CPO will use AI coding assistants. They can write natural language instructions or simple pseudocode, and the assistant will help generate actual code (in Python, JavaScript, or whatever is needed) that uses the above API and LLM to implement the idea.
- **Development UI/Tools (Optional but Helpful):** We expect the vibe coding process might produce small web-based tools for internal use. For instance, the CEO/CPO could quickly code a “Page Preview” web app that shows an original page alongside a transformed page for easy comparison. Another small tool might be a form where they can input a new transformation prompt and apply it to a page on the fly. These mini-apps are not user-facing final products, but rather **internal utilities to speed up development and testing**. They too can be built with the help of the AI assistant.

All these tools combined create a powerful sandbox for experimentation. The heavy lifting (like hosting, caching, LLM computations, etc.) is handled by our backend services and cloud resources, so the vibe coders can focus on the creative aspect of modifying content and designing user experience tweaks.

Step-by-Step Workflow

The vibe coding workflow will proceed in iterative cycles. Each cycle focuses on one hypothesis or idea for improving the user's browsing experience. Below are the typical steps in such a cycle:

1. Define the Hypothesis / Idea

The CEO/CPO starts by **identifying a feature idea or user experience change** to test. This comes from either known customer pain points or creative brainstorming. For example: - *"What if we automatically removed or blurred all discouraging or negative content from a news feed to create a more positive reading experience?"* - *"What if on an e-commerce site, we highlight the best reviews at the top of the page to increase user trust?"* - *"What if we translate certain sections of a page into simpler language for accessibility?"*

They should clearly state the hypothesis: **"We believe that doing X for the user will result in Y benefit."** Starting with a clear hypothesis focuses the experiment.

2. Retrieve the Relevant Pages

Using the **Fetch Page API**, the CEO/CPO (via a small script or command) pulls down the HTML of the page(s) in question. If the page is already cached from prior browsing, it can be retrieved directly from S3. If not, the proxy can fetch it live and then cache it.

- For initial tests, working with a single page (or a few key pages) is fine. For example, to test the "positive news feed" idea, they might fetch the homepage of a news site and a couple of article pages.
- The fetched HTML is then ready for transformation.

(The CTO ensures that at this point the environment is ready: the proxy is running and caching is functioning so that the CEO/CPO can easily get the pages they need.)

3. Use LLM to Modify the Page

With the page content in hand, the CEO/CPO crafts a **prompt or set of instructions for the LLM** to transform the page according to the idea: - They might write a prompt like: "You are an assistant that censors negative content. Remove any paragraphs with negative or sad sentiment from the following HTML page. Output the modified HTML." - Or for the e-commerce reviews example: "Reorder the HTML of this product page so that the customer reviews section appears at the top of the page, just below the product title."

Using the AI coding assistant (or directly using LLM via code), they integrate this prompt into a script: 1. The script sends the prompt and the page HTML to the LLM API. 2. The LLM returns the modified HTML. 3. The script then calls the **Save Page Version API** to store this modified HTML as a new version in the cache (for example, saves it as *PageXYZ_version2*).

This step might require a few iterations: - The CEO/CPO checks the modified HTML output to ensure it matches the intent (e.g., all negative content truly removed, nothing else broken). - If the result isn't right, they adjust the prompt or perform minor edits via code (with AI help) and try again. - Once satisfied, they save the final modified version.

Because of vibe coding, these steps can be done quickly: the CEO/CPO can literally describe changes in natural language and get working code or transformations without writing everything from scratch.

4. Create a Quick Preview (Before/After)

It's often useful at this point to visually verify the changes. The CEO/CPO can **spin up a simple preview UI** to compare original vs modified pages: - One approach is an HTML page with two iframes side-by-

side: one loading the original URL (with cookie set to “original”), and one loading the same URL (with cookie set to the new version ID). - Alternatively, they might use two browser windows: one normal, one with the cookie for the version. But a purpose-built UI is convenient for rapid viewing and making screenshots for presentations.

Using vibe coding, building a preview tool is straightforward: - They describe to the AI assistant: “Create a simple HTML page that takes a URL and version ID, and displays the original and transformed pages side by side for comparison.” - The assistant can generate the HTML/JavaScript needed, which can then be refined if necessary.

This preview step helps ensure the transformed page renders correctly and the changes are as intended: - Are the right elements blurred or removed? - Did the layout break or is it still user-friendly? - Is there any obvious glitch introduced by the LLM’s changes?

If issues are found, the CEO/CPO can tweak the transformation (go back to step 3) before proceeding.

5. Testing via the Proxy in a Real Browser

After confirming the modification looks good in the preview, it’s important to **experience it in a real browsing context**: - The CEO/CPO opens a web browser (with the MITM proxy configured) and sets the special cookie value to the new version ID. - They navigate through the site as an end-user would, while the proxy now serves the transformed pages where available. - This allows testing interactive elements, ensuring links still work, and feeling the overall user experience impact. For example, does removing negative content make the site feel oddly empty, or does it indeed feel more positive and still coherent? Does moving the reviews to the top make the product page more compelling, etc.?

This real browser test is crucial because it simulates exactly what the customer will experience in the demo. Any final adjustments (maybe some CSS fixes or script adjustments) can be discovered here and then corrected by updating the modified HTML (again via the LLM or manual tweak).

6. Apply to Multiple Pages (Scaling the Prototype)

Once the transformation approach works on one page, the next step is to see how it generalizes: - The CEO/CPO can use the LLM or scripts to apply the **same transformation** to a few other pages of the site. For instance, if the idea is “remove negative content,” they should test it on multiple articles; if it’s “reorder reviews,” test on a couple of different product pages. - This might involve looping through a list of URLs, fetching each (via API), applying the same prompt or transformation function, and saving each modified version. - They don’t need to do it for the entire site at this stage, just enough pages to illustrate the feature’s effect and catch any page-specific issues.

By doing this, when it’s time to demo, they can show the feature working on more than one example. It also helps check consistency – maybe the approach needs slight tweaking to handle different page structures.

7. Demo to Customer and Gather Feedback

With the transformed pages ready, the team can now **present the experience to the customer** (this could be in a live meeting or giving them access to the demo environment): - Often the CEO or CPO will lead this, showing a side-by-side comparison: “Here’s your website normally, and now here’s your website with our special filtering/enhancement.” - If possible, letting the customer drive (e.g., giving

them a special link or configuring their browser with the proxy and cookie) is ideal so they can click around and experience it firsthand. - As the customer uses the modified site, the team observes and notes their reactions. Key questions to answer include: - Do they immediately see the value of the modification? - Is there confusion or any negative reaction to how it works? - What do they like or dislike? (For example, "I love that these annoying sections are gone" or "I miss seeing X content") - Does it solve a real problem for them? Would they pay for this capability? - Encourage open feedback. Even if an idea doesn't hit the mark, understanding *why* is valuable (maybe the idea is good but the execution needs tweaking).

The outcome of this step is a clear signal on whether the tested idea is a potential product feature to invest in: - A **strong positive response** (e.g., "This is fantastic, can we have this?") means success – this feature is a candidate for full implementation. - A **lukewarm or negative response** (e.g., "I don't see much use" or "This is neat, but not crucial") means this particular approach might not be the right path, and it's back to the drawing board with a new idea.

8. Iterate or Pivot

Based on the customer feedback: - If the idea was validated, we might still do a few more tweaks or different scenarios to fully flesh out requirements. But largely, a validated idea moves towards planning for production development (outside the scope of this discovery workflow). - If the idea was not validated, the team should analyze the feedback and then **iterate**: - Perhaps modify the approach and test again (maybe the content should be dimmed rather than removed, for example). - Or decide to **pivot to a different hypothesis** altogether if it seems that particular direction isn't promising. - The vibe coding workflow then repeats with the next idea or a refined version of the last idea. Because everything is modular and quick (thanks to the AI-assisted approach), trying a new direction is not a heavy lift.

Throughout multiple cycles, the team should document what was tried and the customer responses. Patterns will emerge that guide where the real value lies.

Definition of Success for this Phase

This vibe coding approach is successful if it enables the team to quickly converge on a desirable product feature-set. Key indicators of success include:

- **Multiple Prototype Experiments Conducted:** The CEO/CPO are able to use the workflow to create a variety of page modifications (at least a few different major ideas) in a short time, without requiring constant hand-holding from the CTO.
- **Live Demo Capability:** We can reliably demonstrate modified web pages via our proxy such that customers feel they are experiencing a real product, not just looking at static mockups.
- **Customer Enthusiasm for a Feature:** At least one of the tested modifications results in clear enthusiasm or commitment from the customer (e.g., "Yes, this solves a big problem for us. We would use this every day."). This is the ultimate validation that we've found a fit.
- **Learning from Failures:** Even for the prototypes that don't get a "wow", we gain concrete understanding of why. For instance, "Customers didn't care about feature X because it didn't address their core need of Y." This learning is captured and used to inform subsequent ideas.
- **Empowerment of Product Team:** The CEO and CPO feel confident that they can dream up an idea and see it realized in a demo the same day. The workflow should reduce friction between having an idea and testing it out. If they are empowered to try things on a whim ("I wonder what happens if we do... Let's test it!"), that's a success of the process itself.

By these measures, we ensure that by the end of this discovery phase, we haven't just built something cool in theory – we have evidence from real users about what features are worth building and investing in. It de-risks the product roadmap significantly.

Benefits of the Vibe Coding Approach

This workflow is intentionally designed to be lightweight and fast-paced, leveraging AI tools. Some benefits of this approach:

- **Rapid Iteration:** Ideas can be built and tested in hours or days instead of weeks. The AI assistants help non-engineers produce working prototypes without waiting for developer resources.
- **Cost-Effective Experimentation:** Because we're using cached pages and offline modifications, we aren't spending time integrating into production systems or scaling prematurely. We only invest big engineering effort after validating an idea.
- **Direct Involvement of Stakeholders:** The people with the closest connection to customer needs (CEO, CPO) are directly crafting the prototype experiences. This removes layers of translation (no long spec documents needed for engineers to implement first), and often produces a better initial alignment with what the customer wants.
- **Creative Freedom:** Vibe coding encourages trying bold or unusual ideas because the barrier to attempt them is low. This can spur innovative approaches that a more heavy-weight process might never consider exploring.
- **Visual, Tangible Output:** Unlike pure strategy discussions or slideware, these prototypes give something real that stakeholders and customers can see and interact with. It's far more impactful to **show** a customer a feature than to tell them about it, especially in early product discovery.

Next Steps After a Validated Prototype

When a prototype is a hit with customers, it doesn't mean the work is done – it's actually the beginning of the next phase: - The CTO and engineering team will take the validated idea and plan how to implement it in the product **for real**. This involves making the transformation happen automatically in real-time for every relevant user request, ensuring the solution scales, and integrating any necessary controls or settings. - The lessons from vibe coding (maybe even some of the code) will guide the implementation. For example, if using the LLM was part of the prototype, the team might need to decide whether to use the LLM in production or to codify the transformation rules in another way for efficiency. - We will also consider any new requirements uncovered during testing (e.g., "It would be even better if it also did X..." from the customer) and include them if feasible. - Finally, we'll define metrics to measure the success of that feature in production (usage, engagement, user satisfaction, etc.), closing the loop from idea to validated product feature.

In parallel, the vibe coding workflow can continue with other ideas or extensions, continually feeding the product pipeline with customer-validated features.

Conclusion

The **vibe coding workflow** for discovering product-market fit allows us to **"feel out" the market by quickly materializing ideas and testing them with real users**. By combining our robust proxy infrastructure with AI-assisted development, we empower the product team to iterate at high speed. This collaborative, experiment-driven approach will guide us to the features that truly matter to

customers, ensuring that the next stages of our product development are grounded in validated insights and poised for success.

The CEO and CPO should now feel equipped to embark on this rapid prototyping journey. With the CTO providing the necessary tools and support, we are ready to explore, experiment, and ultimately find that winning formula that makes customers say, "This is exactly what we need!"
